Kubernetes in Production — Ingress, TLS & Persistent Storage

1. Why Production Kubernetes Is Different

A Spring Boot application running in Minikube may work perfectly during development. For example, a Notes application connected to MySQL might expose:

```
GET  /notes
POST /notes?text=Hello
GET  /whoami
```

Kubernetes can run the application, expose it through a Service and recreate its Pods when they are deleted. However, running successfully is only the first requirement for production.

A production deployment must also answer the following questions:

Production requirement	Kubernetes concept
Users need a domain name and HTTPS	Ingress Controller and TLS
Database files must survive Pod replacement	PersistentVolumes and PVCs
Traffic may increase significantly	Autoscaling
Applications should not have unrestricted cluster access	RBAC and ServiceAccounts
Only approved applications should reach the database	NetworkPolicies and Pod Security Standards
Failures must be diagnosed quickly	Troubleshooting and observability

Core distinction: A running Pod shows that a workload is running. It does not, by itself, prove that the application is secure, durable, reachable or healthy.

2. Ingress Controllers and HTTP/HTTPS Routing

2.1 Why Kubernetes Services are not always enough

Kubernetes provides different ways to access an application:

Service type	Purpose
`ClusterIP`	Exposes a Service inside the cluster
`NodePort`	Makes a Service accessible through a port on each node
`LoadBalancer`	Requests an external load balancer when the infrastructure supports it

Consider an application with User, Order and Payment services. Exposing each service through a different NodePort could result in URLs such as:

```
http://company.com:30080/users
http://company.com:30081/orders
http://company.com:30082/payments
```

Using a separate `LoadBalancer` Service for each application may also increase cost and infrastructure complexity, depending on the provider.

A cleaner public API can use a single domain:

```
https://api.company.com/users
https://api.company.com/orders
https://api.company.com/payments
```

Requests can then be routed to different backend services according to the hostname or URL path.

2.2 Ingress Resource vs Ingress Controller

Ingress is a Kubernetes API resource that declares HTTP/HTTPS routing rules. The resource itself does not process network traffic.

Ingress Controller is software that reads those rules and implements them using traffic-routing infrastructure, often a reverse proxy. Examples include Traefik and NGINX-based controllers.

Component	Responsibility
Ingress resource	Defines hostname and path-routing rules
Ingress Controller	Applies the rules and processes matching traffic
Kubernetes Service	Provides a stable logical destination for the application
Pod	Runs the application

For example:

```
api.company.com/users     → user-service
api.company.com/orders    → order-service
api.company.com/payments  → payment-service
```

The Ingress Controller watches Kubernetes resources, updates its routing configuration when required, and forwards requests to the corresponding backends.

2.3 Complete request flow

For a request to `https://api.example.com/notes`, a common cloud setup works like this:

```
DNS lookup:
api.example.com → Public endpoint IP

Browser (HTTPS request)
        |
        ▼
Cloud Load Balancer
        |
        ▼
Ingress Controller
(host + path matching)
        |
        ▼
notes-app Service (ClusterIP :80)
        |
        ▼
Spring Boot Pod (:8080)
```

1. **DNS resolution:** The browser resolves `api.example.com` to an address associated with the public endpoint.

2. **External load balancer:** Traffic reaches the public load balancer, which forwards it toward the Ingress Controller.
3. **Ingress Controller:** The controller matches the request's host and path against its configured rules.
4. **Kubernetes Service:** The selected backend is the `notes-app` Service. Depending on the controller, traffic may travel through the Service's virtual IP or directly to the Service's ready endpoints.
5. **Spring Boot Pod:** A backend Pod receives the request on port `8080` and returns a response through the established connection.

Important: The cloud Load Balancer and Ingress Controller are not necessarily competing components. An external load balancer commonly provides the public entry point, while the Ingress Controller performs HTTP-level routing inside the Kubernetes setup.

2.4 Service and Ingress configuration

The Spring Boot application can remain private inside the cluster using a `ClusterIP` Service:

```
apiVersion: v1
kind: Service
metadata:
name: notes-app
spec:
type: ClusterIP
selector:
app: notes-app
ports:
-port:80
targetPort:8080
```

The Ingress resource can route the public hostname to that Service:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```yaml
metadata:
name: notes-ingress
spec:
ingressClassName: traefik
rules:
-host: api.example.com
http:
paths:
-path: /
pathType: Prefix
backend:
service:
name: notes-app
port:
number:80
```

This assumes a working Ingress Controller and a matching `traefik` IngressClass are already installed.

Important YAML properties

Property	Meaning
`ingressClassName: traefik`	Selects the Ingress Controller associated with this class
`host: api.example.com`	Matches requests for this hostname
`path: /`	Routes requests under the root path
`pathType: Prefix`	Matches the path and its subpaths
`service.name: notes-app`	Destination Kubernetes Service
`service.port.number: 80`	Port exposed by the Service, not the container port

With `path: /`, URLs such as `/notes`, `/whoami` and `/notes/123` can match the rule.

```
Ingress → Service port 80 → Pod port 8080
```

Ingress routing does **not** automatically remove or rewrite URL prefixes such as `/users` or `/orders`. URL rewriting requires additional implementation-specific configuration.

2.5 Ingress Controller vs API Gateway

Both components can provide routing, load balancing and TLS handling. Depending on the product, both may also support authentication or rate limiting.

An API Gateway may additionally offer API keys, usage plans, developer portals, API management and analytics. The distinction depends on which controller and gateway products are being used, so they are not automatically interchangeable.

2.6 TLS termination

HTTPS protects data in transit using TLS. A TLS connection involves authenticating the server with a certificate and establishing encryption keys before transferring encrypted HTTP data.

TLS termination means that a component accepts the HTTPS connection, performs the TLS handshake and decrypts the request.

When TLS is terminated at the Ingress Controller:

```
Browser
   | HTTPS (encrypted)
   ▼
Ingress Controller
   | HTTP (in this basic setup)
   ▼
Spring Boot Service and Pod
```

The Spring Boot application can continue listening over HTTP on port `8080` because the controller handles client-facing HTTPS.

However, encryption from the browser to the controller does **not** automatically mean the backend connection is encrypted.

For environments requiring encrypted backend traffic, **TLS re-encryption** can be configured:

```
Browser
   | HTTPS
   ▼
Ingress Controller
   | HTTPS
   ▼
Spring Boot Application
```

This requires extra backend TLS and certificate configuration. In some architectures, TLS terminates at the external load balancer instead. The point of termination depends on the infrastructure design.

2.7 Configuring TLS for Ingress

A TLS certificate and corresponding private key can be stored in a Kubernetes Secret:

```
kubectl create secret tls notes-tls \
  --cert=tls.crt \
  --key=tls.key
```

The certificate should be valid for the hostname and trusted by clients. The TLS Secret must be in the same namespace as the Ingress resource.

Add a `tls` section to the Ingress:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
name: notes-ingress
spec:
ingressClassName: traefik
tls:
-hosts:
- api.example.com
secretName: notes-tls
rules:
-host: api.example.com
```

```
http:
paths:
-path: /
pathType: Prefix
backend:
service:
name: notes-app
port:
number:80
```

The controller uses `notes-tls` for the relevant HTTPS hostname. In production, **cert-manager** can automate issuing and renewing certificates through a configured issuer.

2.8 Ingress and Gateway API

The Kubernetes Ingress API remains usable with supported controllers. For new architectures, **Gateway API** offers a newer routing model. The community-maintained **ingress-nginx** controller was retired in March 2026; this does not mean that every NGINX-based controller or every Ingress implementation was retired.

---

3. Persistent Storage: Volumes, PVs, PVCs and StorageClasses

3.1 Why the container filesystem is not sufficient

Our Notes application stores data in MySQL:

```
Spring Boot Pod
      |
      ▼
MySQL Service
      |
      ▼
MySQL Pod
```

When the application receives:

```
POST /notes?text=Hello
```

MySQL writes the corresponding database files to its data directory:

```
/var/lib/mysql
```

A container has a writable filesystem layer, but that layer belongs to the container. If the container is replaced, the new container does not automatically inherit the old container's writable filesystem.

This is why database files need storage independent of the container lifecycle.

3.2 Kubernetes Volumes and `emptyDir`

A **Volume** is storage mounted into a container at a filesystem path. Different types of volumes have different lifecycles.

An `emptyDir` volume is created for a Pod and is useful for temporary files, caches and scratch space:

```yaml
volumes:
-name: temp-storage
emptyDir:{}
```

- If a container restarts **inside the same Pod**, the `emptyDir` data generally remains.
- If the **Pod is deleted**, its `emptyDir` data is removed.

Therefore, `emptyDir` is not suitable as the only storage for production MySQL data.

3.3 PersistentVolume vs PersistentVolumeClaim vs StorageClass

Resource	Meaning	Role
PersistentVolume (PV)	Kubernetes resource representing available persistent storage	Represents the storage allocated to a claim

Resource	Meaning	Role
PersistentVolumeClaim (PVC)	A request for storage	Specifies how much storage and which access mode the application needs
StorageClass	Description of a storage offering	Specifies how and by which provisioner storage should be created

The storage behind a PV might be an Amazon EBS volume, a network-attached disk, an NFS share or another compatible storage system. A PV is a Kubernetes representation of storage, not necessarily the physical disk itself.

The main benefit is **decoupling an application from a particular storage device**. The application requests a storage allocation rather than depending directly on a specific cloud disk ID.

3.4 PVC example

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: mysql-pvc
spec:
accessModes:
- ReadWriteOnce
resources:
requests:
storage: 5Gi
```

This claim requests at least `5Gi` of persistent storage with the `ReadWriteOnce` access mode. Kubernetes can bind it to a compatible existing PV or use dynamic provisioning if the cluster is configured to do so.

3.5 Static vs dynamic provisioning

Static provisioning: An administrator prepares the underlying storage and creates the PV resource ahead of time.

```
Administrator prepares storage
            ↓
Administrator creates PV
            ↓
Developer creates PVC
            ↓
PVC binds to compatible PV
            ↓
Pod mounts the claimed storage
```

Dynamic provisioning: The developer creates a PVC, and a configured storage provisioner creates appropriate underlying storage and its PV automatically.

```
Developer creates PVC
          ↓
StorageClass selects provisioner
          ↓
Provisioner creates storage
          ↓
PV is created and bound to PVC
          ↓
Pod mounts the claimed storage
```

Dynamic provisioning reduces the need to prepare a disk and PV manually for every application.

3.6 What does a StorageClass do?

A StorageClass defines the type of storage available and the provisioner/parameters used to create it.

Illustrative storage offerings:

StorageClass	Intended characteristics	Possible use
`standard`	General-purpose storage	Typical applications
`fast-ssd`	Higher-performance storage	Databases
`shared-storage`	Shared filesystem, where supported	Shared application files

These names are examples; your cluster must actually have the relevant StorageClass and provisioner configured.

A PVC can request a particular class:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: mysql-pvc
spec:
storageClassName: fast-ssd
accessModes:
- ReadWriteOnce
resources:
requests:
storage: 5Gi
```

If `fast-ssd` exists and dynamic provisioning is supported, Kubernetes can arrange suitable storage for this PVC.

3.7 Mounting the PVC into MySQL

A simplified MySQL Deployment can reference the claim:

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: mysql
spec:
replicas:1
selector:
matchLabels:
app: mysql
template:
metadata:
labels:
app: mysql
spec:
containers:
```

```
-name: mysql
image: mysql:8.4
env:
-name: MYSQL_ROOT_PASSWORD
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_ROOT_PASSWORD
ports:
-containerPort:3306
volumeMounts:
-name: mysql-storage
mountPath: /var/lib/mysql
volumes:
-name: mysql-storage
persistentVolumeClaim:
claimName: mysql-pvc
```

This assumes the `mysql-secret` Secret exists and the PVC is created in the same namespace as the Pod.

`volumes` attaches storage associated with the claim to the Pod:

```
volumes:
-name: mysql-storage
persistentVolumeClaim:
claimName: mysql-pvc
```

`volumeMounts` decides where that storage appears inside the container:

```
volumeMounts:
-name: mysql-storage
mountPath: /var/lib/mysql
```

The mount path matters because `/var/lib/mysql` is the MySQL image's default database-file directory.

3.8 What happens when the MySQL Pod is replaced?

```
Before:
MySQL Pod A ── /var/lib/mysql ── PersistentVolume ── Existing data

Pod A is deleted; its Deployment creates Pod B.

After:
MySQL Pod B ── /var/lib/mysql ── Same PersistentVolume ── Existing data
```

When the same PVC remains available and the storage can be attached and mounted, the replacement MySQL Pod can read the existing database files. Kubernetes does not reconstruct the data; it reconnects the workload to storage that already contains it.

3.9 Storage access modes

Access mode	Meaning
`ReadWriteOnce` (RWO)	The volume can be mounted read/write by one node at a time
`ReadOnlyMany` (ROX)	Multiple nodes can mount the volume read-only
`ReadWriteMany` (RWX)	Multiple nodes can mount the volume read/write
`ReadWriteOncePod` (RWOP)	A single Pod can mount the volume read/write, where supported

Important distinction: `ReadWriteOnce` does **not** mean exactly one Pod. Multiple Pods on the same node may be able to access an RWO volume. For single-Pod exclusivity, `ReadWriteOncePod` is the more specific mode where supported.

3.10 Reclaim policy and backups

Two important reclaim-policy behaviors are:

- **Retain:** Keeps the underlying storage available for manual recovery or management after the claim is released.
- **Delete:** Removes the underlying storage when the bound PV is released and deletion is supported by the provisioner.

The outcome depends on the PV and StorageClass configuration.

A PVC provides persistent storage, **not a backup**. A disk failure, accidental deletion or data corruption can still damage the database. Production databases need a separate backup and recovery plan.

---

4. Other Production Topics — Quick Reference

The source script introduces the following topics through a lesson outline. They are kept as concise notes here rather than expanded into a new tutorial.

4.1 Autoscaling: HPA, VPA and Cluster Autoscaler

Mechanism	What it changes	Example
Horizontal Pod Autoscaler (HPA)	Number of Pod replicas	2 → 5 Pods
Vertical Pod Autoscaler (VPA)	CPU/memory sizing of workloads	256Mi → 512Mi
Cluster Autoscaler	Number of worker nodes	2 → 3 nodes

- HPA can scale replicas using utilization metrics. CPU-based utilization targets require CPU requests, and resource metrics are commonly provided through Metrics Server.
- VPA is installed separately and is intended for workload right-sizing. It may conflict with an HPA if both independently control CPU/memory-related scaling.
- Cluster Autoscaler can add node capacity when Pods cannot be scheduled because existing nodes lack sufficient resources, where supported and configured.

Relationship: Increasing Pod replicas does not automatically create more node capacity. An HPA may request extra Pods while the scheduler keeps them `Pending` until capacity becomes available.

4.2 RBAC and ServiceAccounts

Role-Based Access Control (RBAC) defines which identities can perform actions on Kubernetes resources.

Examples of permissions include `get`, `list`, `watch`, `create` and `delete`.

RBAC resource	Role
`Role`	Defines permissions within a namespace
`ClusterRole`	Defines a reusable set of permissions, including cluster-scoped permissions
`RoleBinding`	Grants permissions within a namespace
`ClusterRoleBinding`	Grants permissions at cluster scope

A **ServiceAccount** provides a Kubernetes identity for a workload. A Pod can authenticate to the Kubernetes API through its ServiceAccount, while RBAC determines what it is permitted to do.

Least privilege: Grant only the permissions an application requires. Avoid unnecessary `cluster-admin` privileges and avoid mounting ServiceAccount tokens when not required.

Check an identity's permissions with:

```
kubectl auth can-i get pods
```

4.3 NetworkPolicies and Pod Security Standards

A **NetworkPolicy** controls permitted network traffic to and from selected Pods, provided the cluster uses a network plugin that enforces NetworkPolicy.

- **Ingress policy:** Specifies which traffic can enter selected Pods.
- **Egress policy:** Specifies which destinations selected Pods can access.
- **Default-deny approach:** Deny traffic first, then allow only required communication.

For example, a backend may need access to MySQL on port `3306`, while a frontend should not connect directly to the database.

Pod Security Standards describe security levels for Pod configuration:

Level	Meaning
`Privileged`	Minimal restrictions for trusted workloads
`Baseline`	Prevents common dangerous configurations

Level	Meaning
`Restricted`	Applies stronger workload security requirements

Measures such as running as non-root, preventing privilege escalation and restricting Linux capabilities reduce unnecessary privileges within a container.

Difference: NetworkPolicies govern **who can communicate**. Pod Security Standards govern **how a Pod is permitted to run**.

4.4 Common production failures

Symptom	Check first
`502` / `503` responses	Ingress, Service routing and backend readiness
Pod stuck `Pending`	Node capacity, scheduler events and PVC binding
PVC stuck `Pending`	StorageClass, provisioner and available PVs
HPA does not scale	Metrics availability, CPU requests and HPA configuration
`Forbidden` errors	RBAC bindings and permissions
Backend cannot reach MySQL	Service, database availability and NetworkPolicies
Pod keeps restarting	Application logs, OOMKilled status and health probes

A basic troubleshooting command sequence:

```
kubectl get pods -A
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl get svc,endpointslice,ingress
kubectl get pvc
kubectl get hpa
kubectl auth can-i get pods
```

4.5 Final production checklist

- [ ] External traffic has a suitable routing entry point and HTTPS configuration.
- [ ] Persistent database storage and separate backups are configured.

- [ ] Resource requests, limits and scaling policies are reviewed.
- [ ] Workload permissions follow least-privilege principles.
- [ ] Network traffic is limited to required connections.
- [ ] Pod security requirements are enforced appropriately.
- [ ] Readiness and liveness probes are configured.
- [ ] Rolling updates, rollbacks, logs, metrics and recovery processes are available.

Remember: A `Running` Pod does not necessarily mean that the application is ready to receive traffic or behaving correctly.

Key Concepts at a Glance

Concept	One-line explanation
Ingress	Rules describing how HTTP/HTTPS requests should reach Services
Ingress Controller	Software that actually implements those routing rules
TLS termination	The point where HTTPS is accepted and decrypted
PersistentVolume	Kubernetes representation of persistent storage
PersistentVolumeClaim	An application's request for persistent storage
StorageClass	Rules and provisioner used to allocate a type of storage
Dynamic provisioning	Automatic creation of storage in response to a claim
HPA	Scales Pod count
VPA	Adjusts workload resource sizing
Cluster Autoscaler	Adjusts node count
RBAC	Controls Kubernetes API permissions
ServiceAccount	Identity used by a Kubernetes workload
NetworkPolicy	Controls permitted Pod network traffic
Pod Security Standards	Define acceptable Pod security configurations